

E-graph for IMC

IMCCompiler × e-graph

01

baseline

02

参考论文

03

研究思路

04

代码实现

05

结果分析

baseline

开源工作复现

01



1

基线方法：IMCCompiler

逻辑综合与调度紧耦合的 SIMD 存内计算编译器

High-Quality Iterative Compiler for SIMD Logic-in-Memory Architecture with Tight Coupling of Synthesis and Scheduling

Xingyue Qian¹, Chenyang Lv², Zhezhi He^{2,*}, and Weikang Qian^{1,3,*}

¹University of Michigan-SJTU Joint Institute, ²School of Electronic Information and Electrical Engineering, and ³MoE Key Lab of AI, Shanghai Jiao Tong University, Shanghai, China

Emails: {qianxingyue, lvchenyang, zhezhi.he, qianwk}@sjtu.edu.cn; *corresponding authors

本文面向 SIMD Logic-in-Memory (SIMD LiM) 架构, 提出一种**高质量迭代编译器**。与传统“先逻辑综合、后调度”的串行流程不同, 该方法将**综合与调度紧耦合**: 根据当前调度结果识别内存占用关键区域, 对局部子网重新综合和重调度, 并持续维护 (Size, MF) 的 Pareto 最优设计集合。

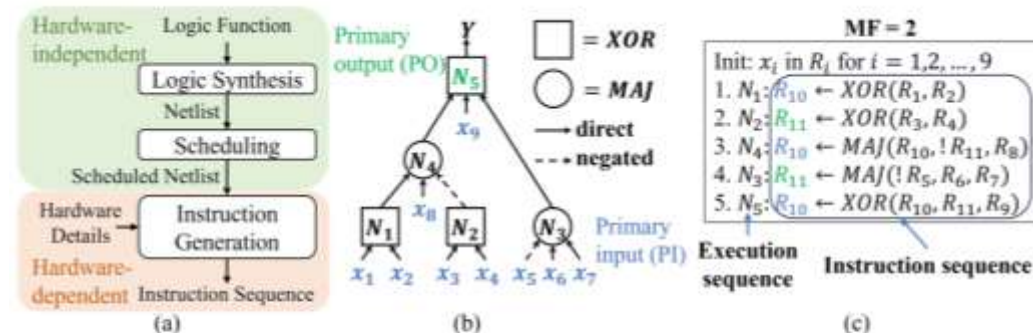


Fig. 1. SIMD LiM compiler: (a) the basic flow; (b) a netlist example. Note that some operations in the figure only have two fan-ins, because we omit the constant 0 for simplicity; (c) an example of execution sequence and instruction sequence.

逻辑综合 $\rightarrow$ 调度 $\rightarrow$ 指令生成

基本流程

综合: 将函数转为 XMG 等支持的逻辑网表
 调度: 决定门/操作的执行顺序
 指令生成: 结合硬件约束生成实际指令序列

Size: 逻辑网表中的操作/门数量

核心指标

MF: Memory Footprint
 执行过程中需要同时保留的最大结果数
 可理解为所需存储行数

1

基线方法：IMCCompiler

论文创新点

综合与调度的紧耦合迭代优化

整体流程

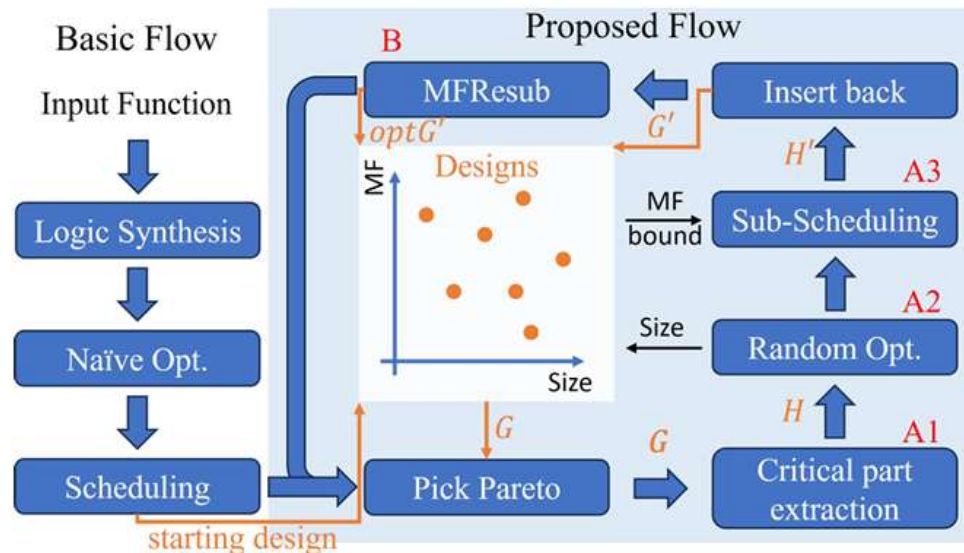
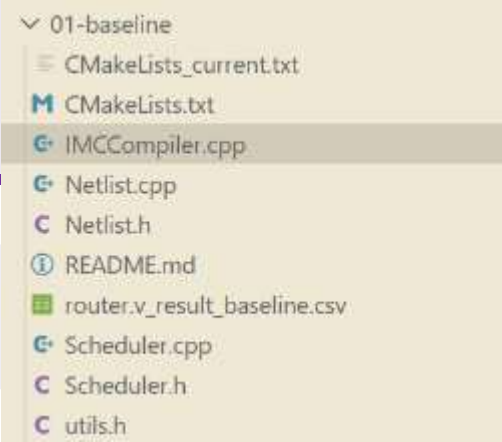


Fig. 3. The flow of our iterative LiM compiler.

1. 从串行流程变为闭环迭代流程：根据调度结果继续指导综合，形成反馈闭环；
2. 以 (Size, MF) Pareto 前沿为目标：保留二者不可互相支配的设计点，供不同硬件资源约束下选择；
3. 只优化 MF 关键子网：从当前设计的 MF 峰值附近提取关键子网，进行局部逻辑优化和重新调度，再回填到完整网表，提高迭代效率。

1 Baseline的代码实现



NewDSE()是baseline的核心入口



文件/类	核心函数	作用
IMCCompiler.cpp	main()	读取 benchmarks.txt, 创建 Scheduler, 读入电路, 启动 NewDSE()
Scheduler.cpp	NewDSE()	baseline 的总调度器: 选择设计、提取关键子网、优化、重新调度、更新 Pareto
Netlist.cpp	ConfigMF()	根据当前执行顺序模拟中间结果的占用, 计算 MF; 并负责子网回填等网表操作

1 NewDSE(): 根据 MF 定位并优化关键子网



1、MF 是执行过程中的峰值存储需求

某时刻仍需保留的中间结果最多，说明该区域最值得优化。

2、不直接反复优化全网

全网综合与调度代价高；baseline 从 MF 峰值附近提取关键子网，以降低每轮迭代成本。

3、局部优化仍以全局结果验收

子网优化、重新调度后必须回填完整网表，再计算全局 (Size, MF) 并更新 Pareto 前沿。

```
Scheduler.cpp x
01-baseline > Scheduler.cpp
997 //update pareto
998 }
999
1000 void Scheduler::NewDSE()
1001 {
```

baseline 的关键思想：用调度结果定位“值得综合优化”的局部逻辑。

1 baseline 的两类优化：逻辑化简与 MF 定向重代换

一、局部逻辑优化

- 主要目标：降低Size
- 对关键子网应用 resyn2、rewrite、resubstitution 等操作，寻找功能等价但更小的实现。



普通逻辑优化主要关注“门能否变少”；MFResub 则利用调度信息关注“中间结果能否更早被释放”。

二、MFResub

- 主要目标：降低MF，也可能同时降低Size
- 利用当前调度顺序，尝试让某些中间结果更早失效、释放存储行。

MFResub

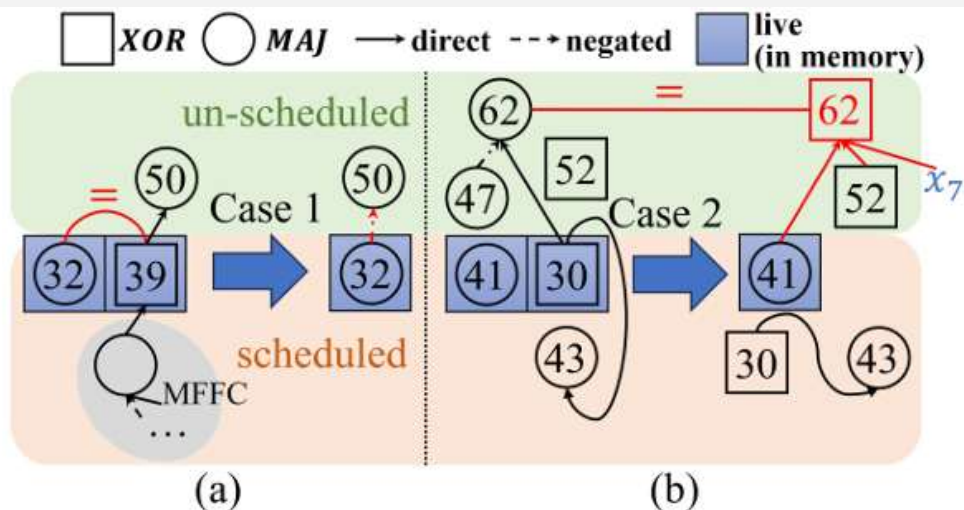


Fig. 5. The two cases of MF-oriented resubstitution: (a) case 1 where there exist two functionally equivalent live operations, 32 and 39; (b) case 2 where we can resubstitute the only un-scheduled fan-out of operation 30 to make operation 30 no longer live.

- ◆ Case 1: 两个仍在内存中的节点功能等价，删除其中一个；
- ◆ Case 2: 重写后续节点，使某个中间结果提前失效、释放。

baseline 的局部重写

能否探索更大的等价逻辑空间？

e-graph: 同时保存多种等价表达

IMC 后端按全局 (Size, MF) 评估并选择

参考文献

E-GRAPH等价空间探索

02

2 e-graph: 如何系统地保留逻辑等价实现

1.e-node

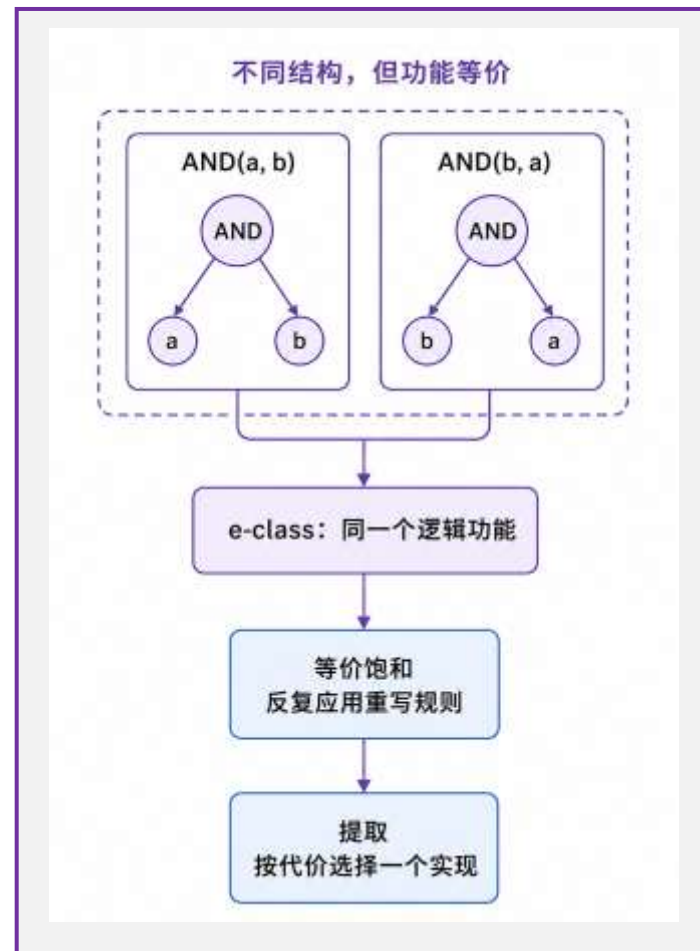
一种具体逻辑表达或门结构
例如 $\text{AND}(a,b)$ 、 $\text{AND}(b,a)$

2. e-class

把功能等价的 e-node 放在一起
因此它表示 “同一个逻辑功能的多种实现”

3. equality saturation + extraction

持续应用等价重写规则扩展候选
最后按代价函数提取一个实现，而不是每改一步就立刻提交



传统重写通常 “找到一种改法就提交”；e-graph 则先保留多种等价改法，最后再选择。

eLogic: 面向多数-反相图(MIG)的 e-graph 逻辑重写框架

eLogic的局部优化思想

eLogic: An E-Graph-based Logic Rewriting Framework for Majority-Inverter Graphs

Rongliang Fu¹, Wei Xuan², Shuo Yin¹, Guangyu Hu², Chen Chen³, Hongec Zhang³, Bei Yu¹, Tsung-Yi Ho¹¹CUHK ²HKUST ³HKUST-GZ

rlfu@cse.cuhk.edu.hk

Abstract—Majority-Inverter Graph (MIG) emerges as a promising data structure for logic optimization and synthesis, offering a more compact representation for logic functions compared to traditional AND/OR-Inverter graphs. Consequently, the MIG finds widespread application in digital circuit design, particularly in quantum circuits and superconducting adiabatic quantum-flux-parametron logic circuits. Currently, logic optimization techniques for MIG mainly fall into two categories: (i) logic rewriting with predefined more compact sub-structures and (ii) logic resubstitution with already existing logic in the Boolean network. However, the inherent complexity of MIG logic and the limitation imposed by the input scale of sub-structures significantly impact the performance of these methods. To address these challenges, this paper proposes eLogic, a novel depth-oriented MIG logic rewriting framework using e-graphs, to minimize the depth and size of MIG. The eLogic utilizes the e-graphs, a data structure for efficient computation with equalities between terms, to minimize the depth and size of the cone delimited by the cut. The experimental results on the EPFL benchmark demonstrate the effectiveness of eLogic. It is noteworthy that eLogic is open-sourced on <https://github.com/Flians/eLogic>.

Index Terms—Logic synthesis, logic rewriting, majority-inverter graph, e-graph.

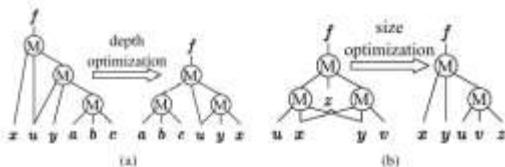


Fig. 1. MIG logic optimization usually requires a large cut size. (a) shows the depth optimization for a 6-cut of node f . (b) shows the size optimization for a 5-cut of node f . $\textcircled{M}$ represents a node with a 3-input majority function.

[15]. Boolean resubstitution techniques focus on optimizing a specific node by replacing it with a new function derived from its local neighborhood. However, these techniques are limited in their ability to explore the solution space, as they only consider functions that can be formed using the existing nodes in the local neighborhood. On the other hand, Boolean rewriting techniques utilize a set of predefined rewriting rules to transform subgraphs into more efficient representations. These

开源实现: <https://github.com/Flians/eLogic>

K-cut 枚举

计算该窗口的 MFFC

在小窗口内运行 e-graph

提取候选并安全替换, 再继续枚举

eLogic 启发了窗口化架构: cut → MFFC → 局部 e-graph → 安全回填

E-morphic: 用于逻辑综合结构探索的可扩展等价饱和方法

E-morphic: Scalable Equality Saturation for Structural Exploration in Logic Synthesis

Chen Chen^{*}, Guangyu Hu^{*†}, Yuzhe Ma, Hongce Zhang[†]
HKUST & HKUST(GZ)
{cchen099, yuzhema, hongcezh}@connect.hkust-gz.edu.cn
ghuae@connect.ust.hk

Cunxi Yu
University of Maryland, College Park
cunxiyu@umd.edu

Abstract—In technology mapping, the quality of the final implementation heavily relies on the circuit structure after technology-independent optimization. Recent studies have introduced equality saturation as a novel optimization approach. However, its efficiency remains a hurdle against its wide adoption in logic synthesis. This paper proposes a highly scalable and efficient framework named E-morphic. It is the first work that employs equality saturation for resynthesis after conventional technology-independent logic optimizations, enabling structure exploration before technology mapping. Powered by several key enhancements to the equality saturation framework, such as direct e-graph-circuit conversion, solution-space pruning, and simulated annealing for e-graph extraction, this approach not only improves the scalability and extraction efficiency of e-graph rewriting but also addresses the structural bias issue present in conventional logic synthesis flows through parallel structural exploration and resynthesis. Experiments show that, compared to the state-of-the-art delay optimization flow in ARC, E-morphic on average achieves 12.54% area saving and 7.29% delay reduction on the large-scale circuits in the EPFL benchmark.

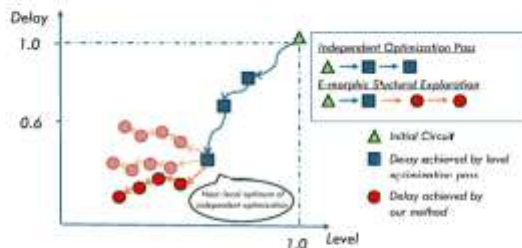


Fig. 1: E-morphic: parallel structural exploration for delay finetuning improves the circuit performance.

[9], datapath optimization [10], multiplier optimization [11], etc. Among these prior works, E-Syn [12] first applied e-graph rewriting to Boolean logic optimization and demonstrated

1、论文解决的问题

传统逻辑重写通常命中一条规则便立即提交，容易错过“先改变结构、再得到更优实现”的路径。

2、论文核心思想

使用 equality saturation 在 e-graph 中同时保留并扩展多种等价结构，最后再按代价提取实现。

3、采用的思想

e-graph 生成多个等价候选、最后再选择，项目的 Rust 程序中，使用 egg 库的 Runner，对局部子网执行等价饱和。

项目未直接复现 E-morphic 的完整搜索策略与提取器，只是参考 e-graph 进行逻辑结构探索的方法依据。

SkyEgg: 利用 e-graph 联合优化硬件实现选择与调度

SkyEgg: Joint Implementation Selection and Scheduling for Hardware Synthesis using E-graphs

Youwei Xiao
Peking University
China

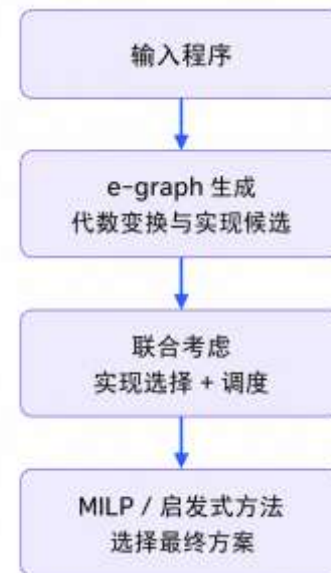
Yuyang Zou
Peking University
China

Yun Liang
Peking University
China

1、论文解决的问题

“先选择一种硬件实现、再做调度”的串行流程，忽略了实现选择与调度之间的相互影响，因此可能错过更优设计。

2、论文方法



3、区别

该工作和本项目的不同：面向 FPGA，使用 MILP/ASAP；因此项目没有使用或复现 SkyEgg 的工作。

启发：候选的价值应由下游实现与调度代价决定，而非只看局部逻辑代价。

A Novel Basis for Logic Rewriting

面向 MIG 的代数重写基础

A Novel Basis for Logic Rewriting

Winston Haaswijk*, Mathias Soeken*, Luca Amarù†, Pierre-Emmanuel Gaillardon‡, Giovanni De Micheli*

*Integrated Systems Laboratory, EPFL, Lausanne, VD, Switzerland

†Design Group, Synopsys Inc., Mountain View, CA, USA

‡Laboratory for NanoIntegrated Systems, The University of Utah, Salt Lake City, UT, USA

基础化简规则

- ◆ 交换律: $M(a,b,c) = M(b,a,c)$
- ◆ 幂等律: $M(a,a,b) = a$
- ◆ 互补律: $M(a, \neg a, b) = b$

结构变换规则

- ◆ 结合律 associ
- ◆ 分配律 distri
- ◆ 相关性律 relevance

为 maj3 的结构重写规则提供代数基础

Structural Rewriting in XOR-Majority Graphs

面向 XMG 的 MAJ-XOR 重写

Structural Rewriting in XOR-Majority Graphs

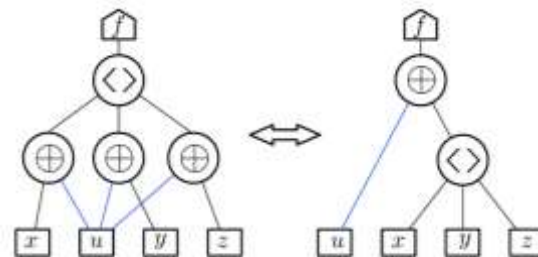
Zhufei Chu¹ Mathias Soeken² Yinshui Xia¹ Lunyao Wang¹ Giovanni De Micheli²¹Faculty of Electrical Engineering & Computer Science, Ningbo University, Ningbo, China²École Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland

Figure 2: XMG demonstration of Theorem 1.

THEOREM 1. XOR operation distribute over MAJ function.

$$\langle xyz \rangle \oplus u = \langle (x \oplus u)(y \oplus u)(z \oplus u) \rangle$$

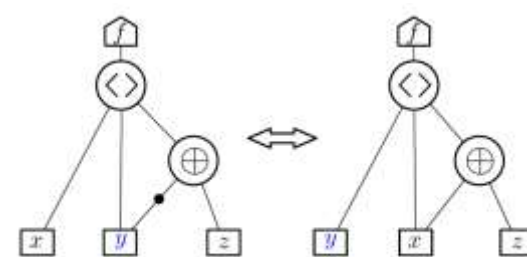


Figure 3: XMG demonstration of Theorem 2.

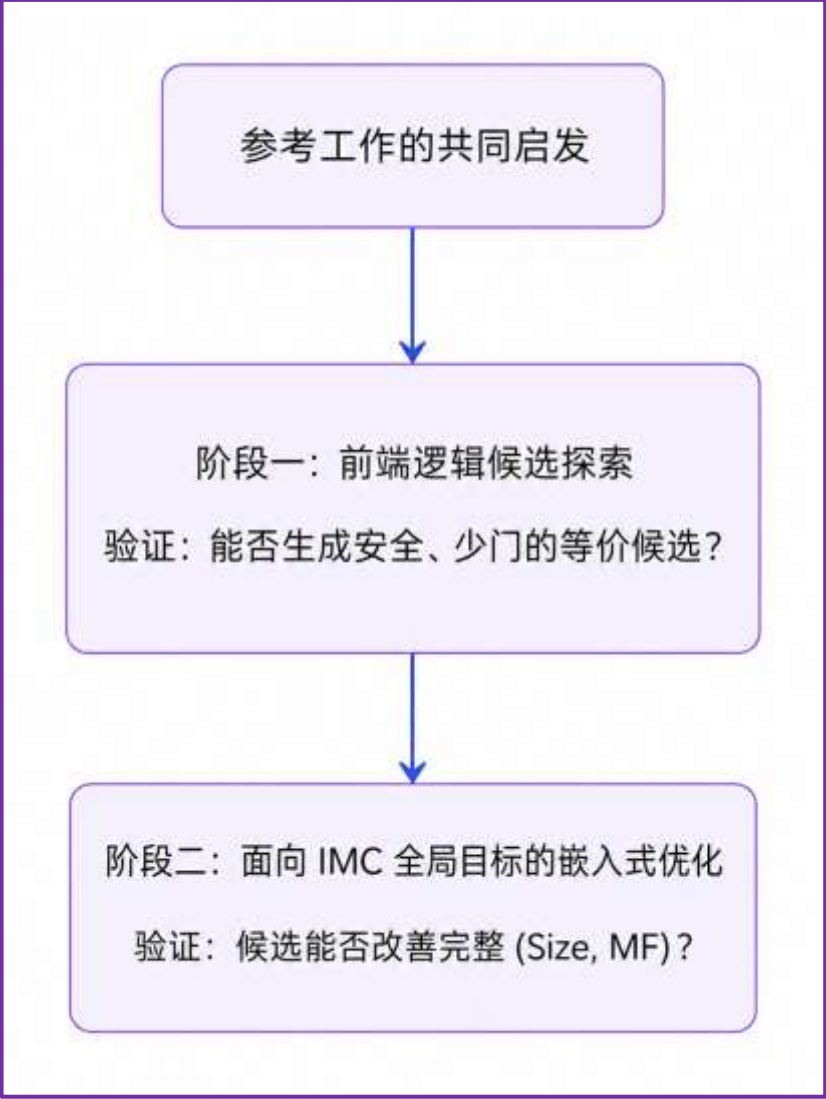
THEOREM 2. Complementary associativity rule exists in MAJ-XOR operations.

$$\langle xy(\bar{y} \oplus z) \rangle = \langle xy(x \oplus z) \rangle \quad (14)$$

XMG 相对 MIG 的额外优化机会

2 Conclusion

参考工作	核心思想	项目
eLogic	cut + MFFC 中运行局部 e-graph 安全替换	阶段一的局部候选生成框架；后续被阶段二继承
E-morphic	等价饱和：保留多个候选，最后选择	阶段一/二中使用 egg 生成局部等价候选
SkyEgg	候选价值应考虑下游实现与调度	阶段二按全局 (Size, MF) 评估候选
MIG/XMG 规则	MAJ 与 MAJ-XOR 的功能等价变换	阶段一/二共用的 Rust 重写规则集



03

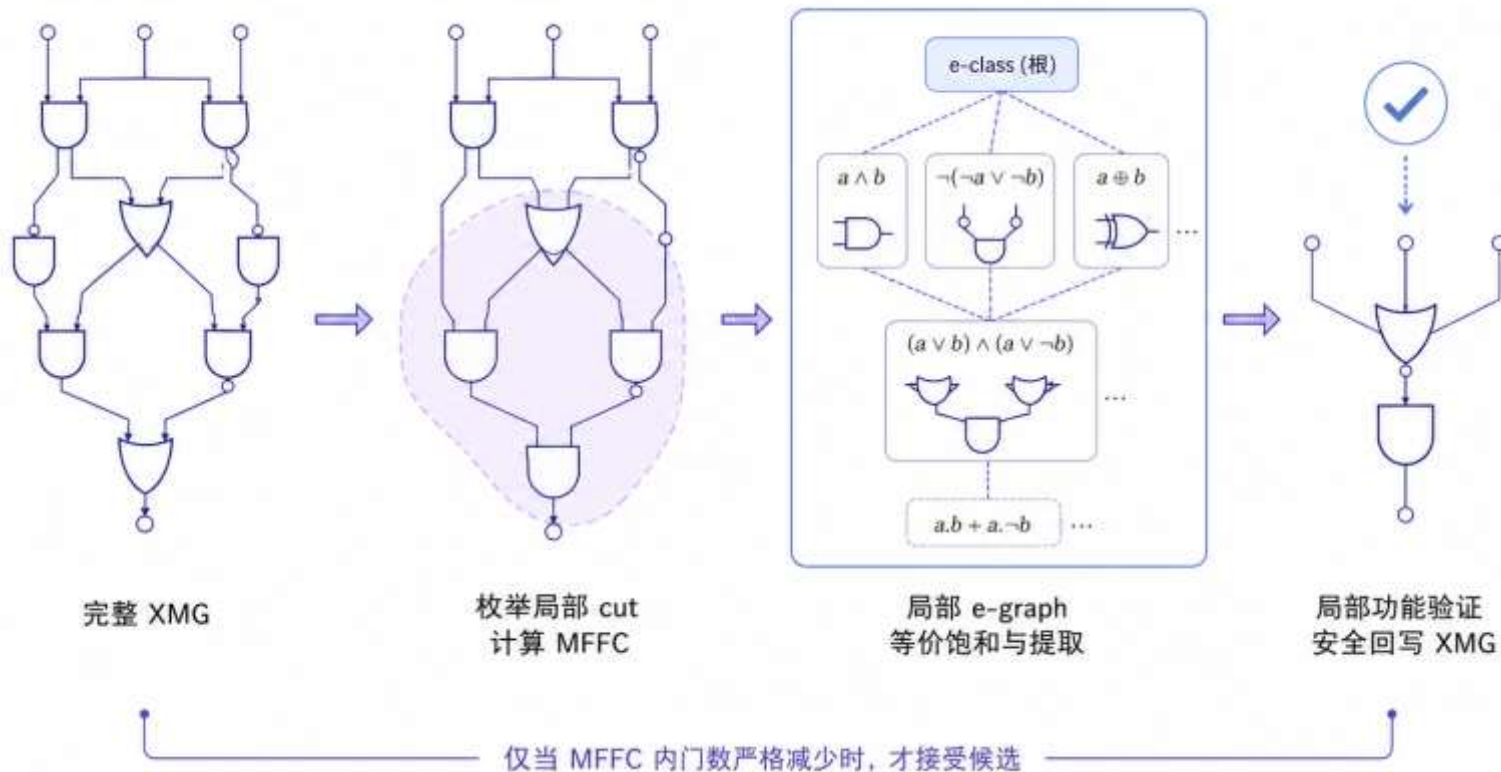
研究思路演进

局部逻辑候选 → 全局 IMC 优化

3

阶段一：前端逻辑候选探索

不改变 IMC 调度器，e-graph 先在 XMG 局部产生安全、真实少门的等价逻辑候选



1. 优化窗口：cut+MFFC

cut 限制 e-graph 的搜索规模；
MFFC 确保替换后统计的门确实可以从全网删除，避免多扇出共享门导致的虚假收益。

2. 正确性：局部功能等价验证

回写完成后，对原始/优化完整网表进行
2000 组随机仿真，比较全部输出。

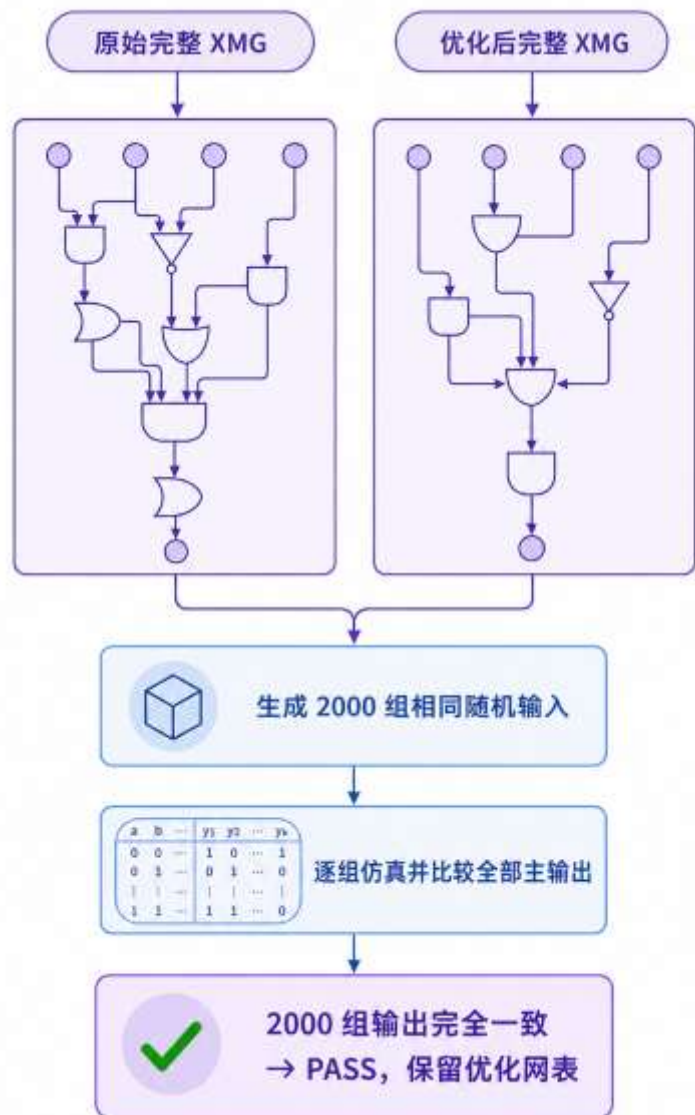
3. 阶段一的选择标准：局部门数

只有局部逻辑门数严格减少，才回写
XMG。
尚未把 (Size, MF) 用作最终选择标准。

阶段一验证“能生成什么候选”，但尚不能回答“候选是否真的改善 IMC 映射”。

3

阶段一的正确性保障：局部功能等价验证



- ◆ 对原始 XMG 与优化后 XMG 使用同一组随机输入;
- ◆ 共进行 2000 组输入仿真;
- ◆ 逐组比较所有主输出;
- ◆ 全部一致则输出 PASS, 保留优化网表。

3 阶段一端到端结果：局部少门不等于 IMC 全局最优

实验结果

电路	E-graph前端结果	原始电路的IMC Pareto前沿	e-graph 后电路的 IMC Pareto 前沿	结论
Ctrl	174 → 148, 少26门	(81, 25)	(99, 25)	同一 MF 下 Size 增加 18, 变差
int2float	260 → 253, 少7门	(200, 18)、(202, 16)、 (211, 15)、(260, 14)	(209, 19)、(215, 18)、 (223, 17)、(253, 15)	优化后所有点都被原始前沿支配

结论及改进

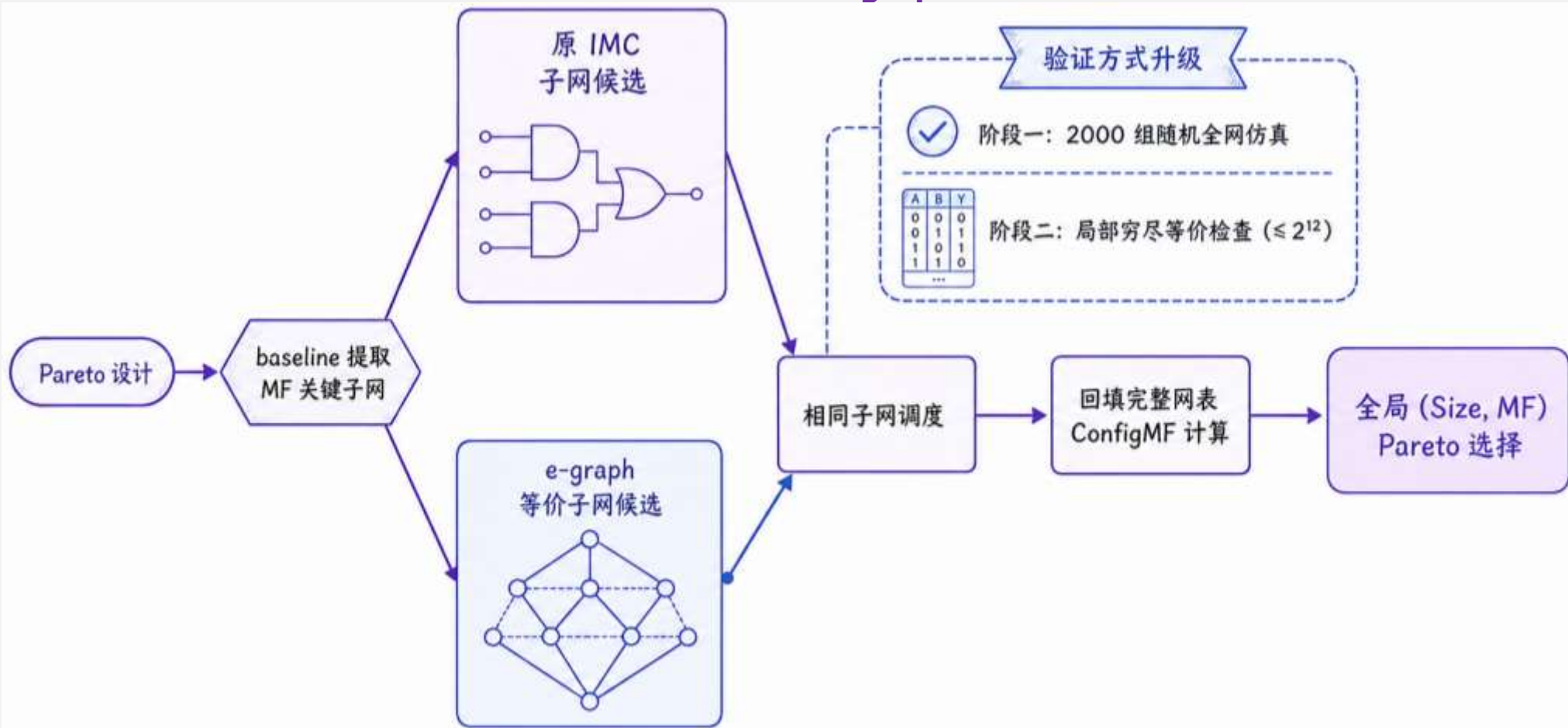
e-graph 前端确实降低了原始 XMG 门数，但会改变 IMCCompiler 后续综合与调度可利用的结构；在已测的 ctrl 与 int2float 上，完整 (Size, MF) 均没有改善，反而退化。

因此，**阶段二**不再仅按局部门数选择候选，而是将候选放入 IMC 子网优化流程，按完整 (Size, MF) 评价。

3

阶段二：将 e-graph 嵌入 IMCCompiler 的子网优化环节

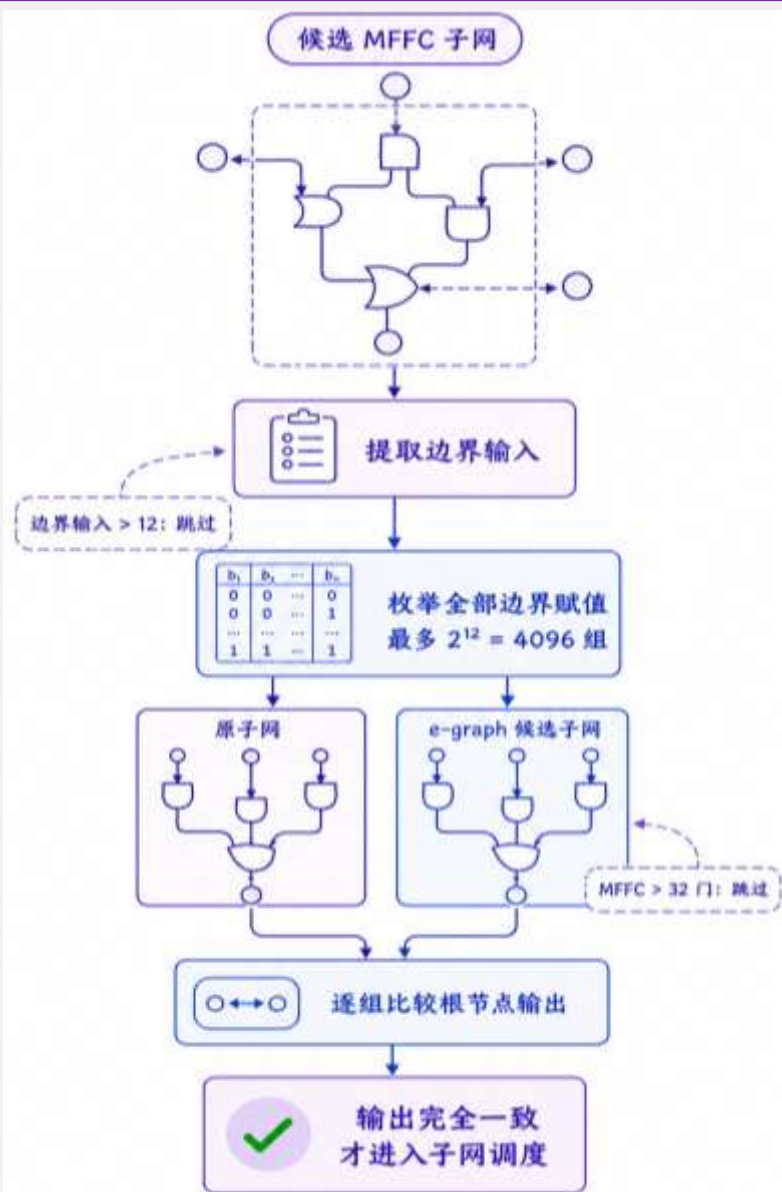
不再先改完整网表，而是在 baseline 已选出的待优化子网内生成 e-graph 候选，与原 IMC 候选一起接受相同调度和全局评价。



阶段二不预设“少门就更好”，只有完整 (Size, MF) 改善的候选才能进入 Pareto 集合。

3

阶段二的正确性保障：局部功能等价验证



阶段二不再依赖“随机输入恰好没发现错误”，
而是在可控局部窗口内用穷尽枚举保证候选等价。

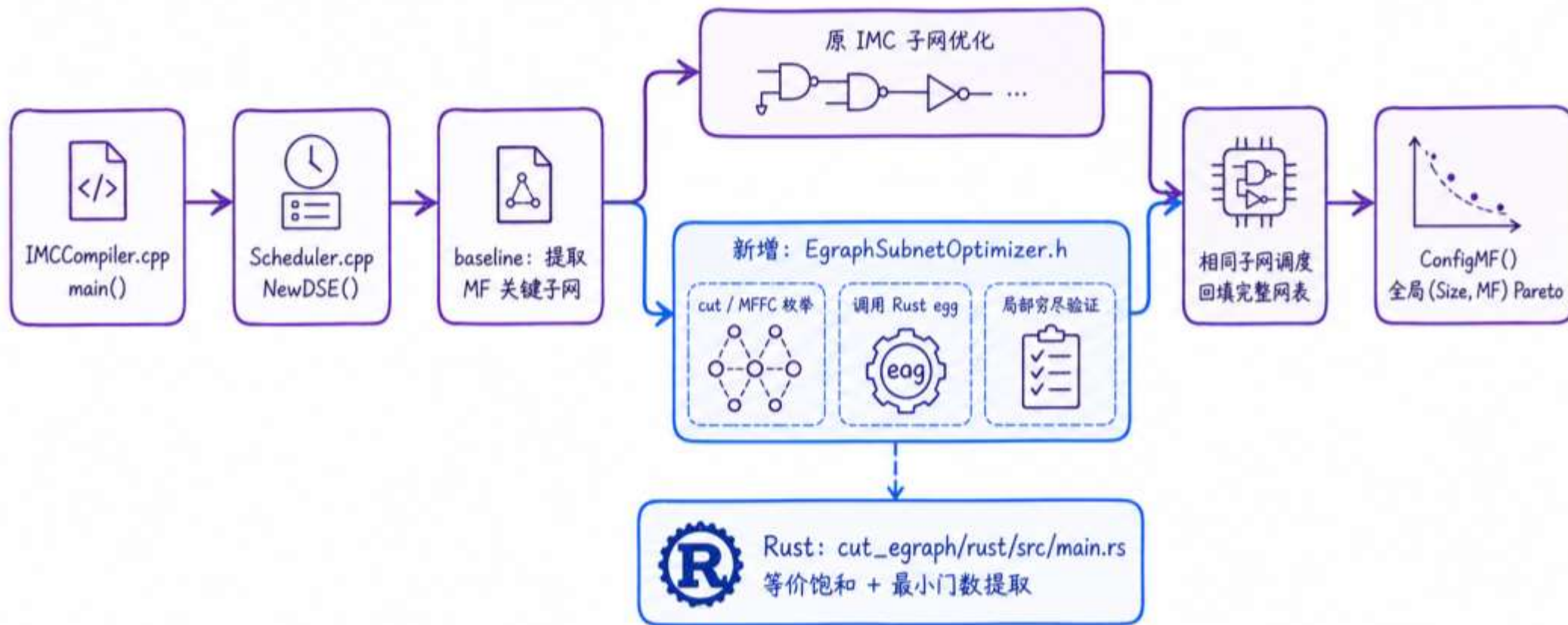
- ◆ MFFC 内部门数不超过 32;
- ◆ 边界输入数不超过 12;
- ◆ 最多枚举 $2^{12} = 4096$ 组输入;
- ◆ 任一输入下输出不同、或超出上限：直接拒绝候选;
- ◆ 证明局部函数真值表等价，而非全网 SAT 形式证明。

代码实现

04

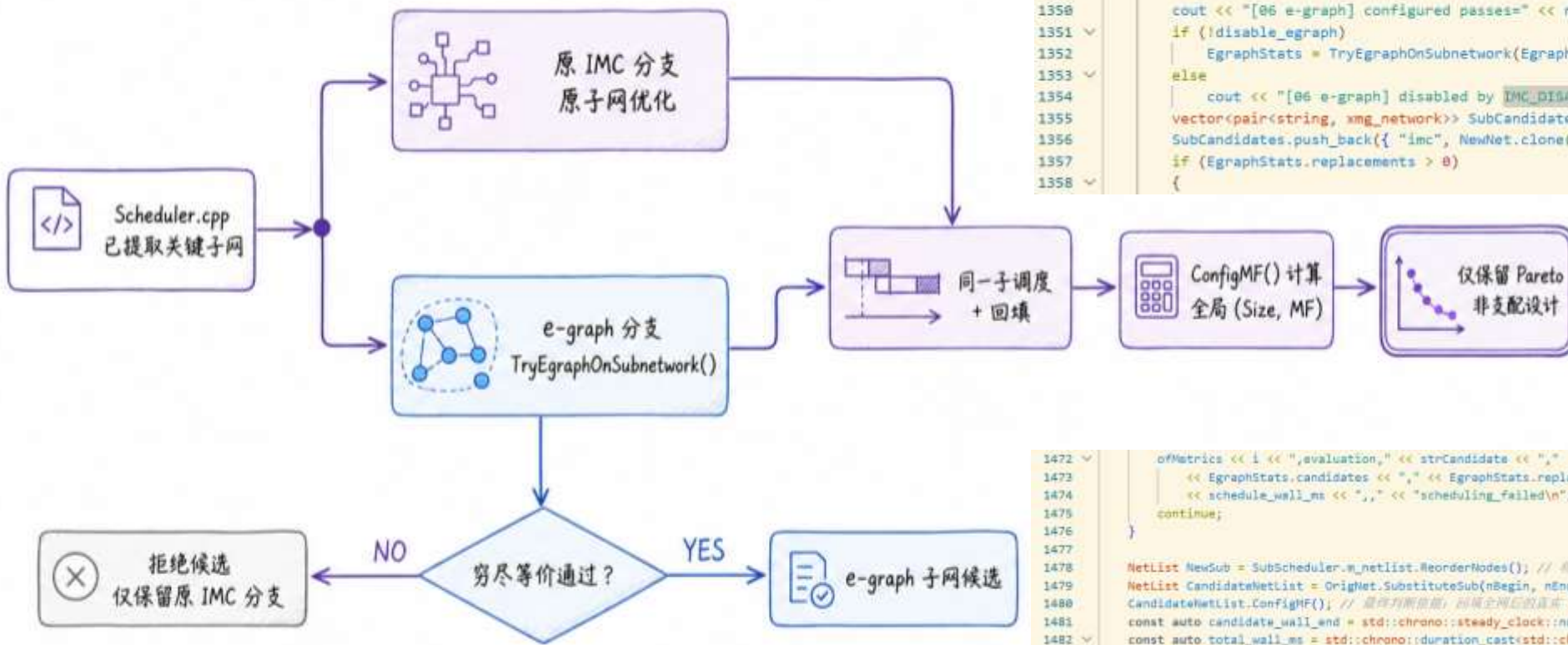
4

代码总览：在 baseline 上新增 e-graph 子网优化模块



baseline 的 `NewDSE()` 主流程保持不变，新增模块只负责生成并验证 e-graph 子网候选

4 Scheduler.cpp: 原 IMC 与 e-graph 候选的 A/B 接入



生成 e-graph 子网候选

```

1345 // 显式关闭 e-graph, 供同一输入下的 baseline
1346 // 路径隔离测试使用, 默认不关闭。
1347 EgraphSubnetStats EgraphStats; // 接收生成阶段的结构, 证明与跳过统计。
1348 const bool disable_egrph = std::getenv("IMC_DISABLE_EGRAPH") != nullptr; // 隔离 A/B 时则关闭 e-graph。
1349
1350 cout << "[06 e-graph] configured passes=" << nEgraphReplacements << "\n";
1351 if (!disable_egrph)
1352 | EgraphStats = TryEgraphOnSubnetwork(EgraphCandidate, nEgraphReplacements);
1353 else
1354 | cout << "[06 e-graph] disabled by IMC_DISABLE_EGRAPH\n";
1355 vector<pair<string, xmg_network>> SubCandidates; // 同一轮里要使用相同下游管道评估的子网候选。
1356 SubCandidates.push_back({"imc", NewNet.clone()}); // 无论 e-graph 成功与否, 原 IMC 候选始终存在。
1357 if (EgraphStats.replacements > 0)
1358 {

```

```

1472     ofMetrics << i << ",evaluation," << strCandidate << ",  

1473         << EgraphStats.candidates << ",<< EgraphStats.replacements << ",,,,,,,,,,  

1474         << schedule_wall_ms << ",," << "scheduling_failed\n";  

1475     continue;  

1476 }  

1477  

1478 NetList NewSub = SubScheduler.w_netlist.ReorderNodes(); // 将子网调度字串化为新的局部 Node 顺序。  

1479 NetList CandidateNetList = GrigNet.SubstituteSub(nBegin, nEnd, NewSub, vecTIOOrigIndex, vecPOOrigIndex); // 用共同边界映射创建完整网络  

1480 CandidateNetList.ConfigMF(); // 最佳判例依据: 网络全局后验成本 Size: MF. Cross.  

1481 const auto candidate_wall_end = std::chrono::steady_clock::now();  

1482 const auto total_wall_ms = std::chrono::duration_cast<std::chrono::milliseconds>{  

1483     candidate_wall_end - candidate_wall_start}.count();  

1484 cout << "[06 " << strCandidate << "] global: #nodes = "

```

回填完整网表后计算全局指标

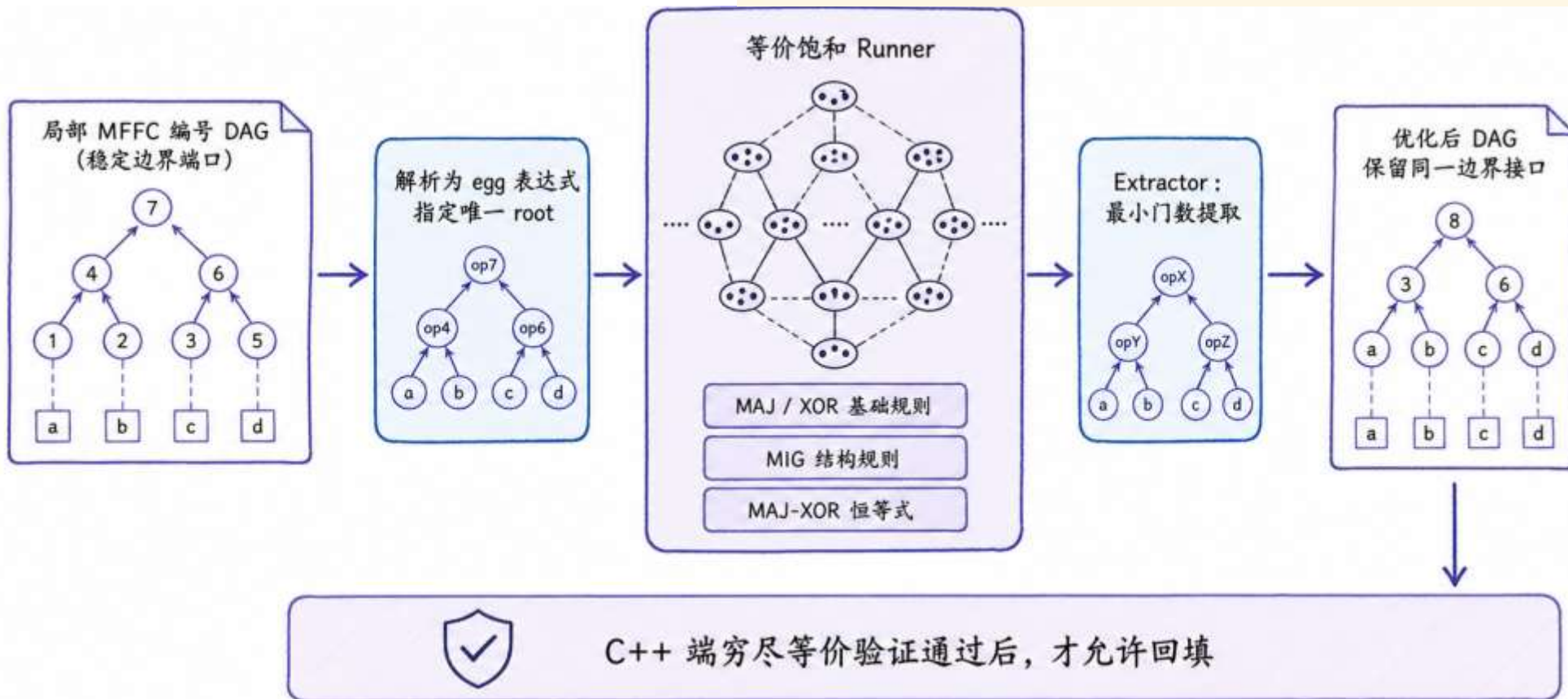
验证失败或没有严格少门候选时, e-graph 分支自然退出, 原 IMC 分支不受影响

4

Rust egg: 局部等价饱和与最小门数候选提取

```
//-----XMG-特有-MAJ-XOR-恒等式 (Structural-Rewriting-in-XMG, ASP-DAC-2019) ---  
//-----这是-XMG-相对-MIG (eLogic) 的独特优势。2-输入-XOR-用-xor3(x,u,0)-表达。  
//-----Theorem-1-(Eqn-10): XOR-分配-over-MAJ。RL-方向-4门+2门(减门关键)。  
//-----maj3(x@u, y@u, z@u) = maj3(x, y, z) @ u  
rwl("maj-xor-distri"; "(maj3(xor3 ?x ?u 0) (xor3 ?y ?u 0) (xor3 ?z ?u 0))" => "(xor3 (maj3 ?x ?y ?z) ?u 0)"),  
rwl("maj-xor-distri-flip"; "(xor3 (maj3 ?x ?y ?z) ?u 0)" => "(maj3 (xor3 ?x ?u 0) (xor3 ?y ?u 0) (xor3 ?z ?u 0))"),  
//-----Theorem-2-(Eqn-14): 互补结合律。处理-y/0-重汇聚。移除反相器。  
//-----maj3(x, y, y@z) = maj3(x, y, x@z)  
rwl("maj-xor-assoc"; "(maj3 ?x ?y (xor3 (not ?y) ?z 0))" => "(maj3 ?x ?y (xor3 ?x ?z 0))"),  
rwl("maj-xor-assoc-flip"; "(maj3 ?x ?y (xor3 ?x ?z 0))" => "(maj3 ?x ?y (xor3 (not ?y) ?z 0))"),  
... ]  
}
```

XMG 特有的 MAJ-XOR 重写规则



部分日志

Original saved log excerpt: int2float_k1_license_fixed.out.log

```
[06 e-graph] subnet gates 234 -> 233, replacements=1, candidates=982, depth=13, max_fanout=6
[06 imc] global: #nodes = 237, MF = 18
[06 egraph] global: #nodes = 236, MF = 18
```

Source: D:\DESKBOOK\0810\06\testdata\int2float_k1_license_fixed.out.log
This is raw terminal output from the saved log file.
请按任意键继续. . .

Original saved log excerpt: ctrl_k1_seed1_metrics.out.log

```
[06 e-graph] subnet gates 129 -> 127, replacements=1, candidates=319, depth=9->9, max_fanout=6->6, selected_mffc_total=3
, selected_boundary_total=5, selected_boundary_max=5, selected_local_gain_total=1
[06 imc] global: #nodes = 137, MF = 25, schedule_wall_ms=4590, total_wall_ms=4592
[06 egraph] global: #nodes = 135, MF = 25, schedule_wall_ms=2010, total_wall_ms=2011
```

Source: D:\DESKBOOK\0810\06\testdata\ctrl_k1_seed1_metrics.out.log
This is raw terminal output from the saved log file.
请按任意键继续. . . |

Original saved log excerpt: int2float_k1_seed2_license_fixed.out.log

```
[06 e-graph] subnet gates 239 -> 237, replacements=1, candidates=990, depth=17, max_fanout=6
[06 imc] global: #nodes = 242, MF = 17
[06 egraph] global: #nodes = 240, MF = 18
```

Source: D:\DESKBOOK\0810\06\testdata\int2float_k1_seed2_license_fixed.out.log
This is raw terminal output from the saved log file.
请按任意键继续. . . |

文件目录

- 8-25
 - papers
 - baseline_IMCCompiler.pdf
 - eLogic_DATE2026.pdf
 - SkyEgg.pdf
 - stage1
 - examples
 - results
 - source
 - README.md
 - stage2
 - results
 - source
 - testdata
 - README.md
 - README.md

05

当前阶段实验结果

5

阶段二的选择准则：从局部门数到全局 (Size, MF)

	阶段一：前端候选探索	阶段二：嵌入式优化
候选位置	完整网表进入IMC前	Baseline提取的关键子网内
候选生成	e-graph 产生少门等价实现	e-graph 产生已验证等价子网
首要判断	局部门数是否严格减少	回填后完整 (Size, MF) 是否改善
调度评价	不参与候选选择	原IMC与 e-graph 候选走相同调度
最终接受	前端直接输出优化网表	仅保留全局Pareto 非支配设计

局部门数只用于生成或排序候选；候选的最终价值由完整网表的 Size 与 MF 共同决定。

5

阶段二实验设置：受控 A/B 对照

项目	设置
 输入电路	int2float、ctrl、router
 对照分支	原 IMC 子网候选 vs. e-graph 等价子网候选
 公平性	相同子网调度、回填与 ConfigMF
 随机控制	固定 IMC_SEED；记录不同 seed 样本
 调度配置	IMC_THREADS = 48；IMC_RUNS = 2
 替换次数	k = 1 为主；补充 k = 2
 结果指标	完整网表的 (Size, MF) Pareto 关系

5 阶段二实验结果：支配、权衡与无候选

电路	Seed	K	原IMC	E-graph	全局关系
Int2float	1	1	(237, 18)	(236, 18)	支配: Size -1, MF不变
Int2float	2	1	(242, 17)	(240, 18)	权衡: Size -2, MF +1
Int2float	3	1	(219, 20)	无候选	无验证改进
Int2float	1	2	(237, 18)	(235, 19)	权衡: Size -2, MF +1
Ctrl	1	1	(137, 25)	(135, 25)	支配: Size -2, MF 不变
router	1	1	(237, 22)	无候选	无验证改进

实验结论

阶段二能扩展 IMC 的设计空间：可能产生支配点、形成 Size/MF 权衡，或因没有安全候选而保持原 IMC 结果。但当前证据不足以宣称普遍提升。